OpenClaw API-KEY Configuration

OpenClaw API-KEY Configuration

1. Course Content
2. Preparation
3. API-KEY Application
4. Configure OpenClaw API-KEY
    - 4.1 Configuration Methods
    - 4.2 Quick Configuration
        - 4.2.1 Configure baseurl
        - 4.2.2 Fill in apiKey and baseUrl
5. View Connected Model Status
    - 5.1 View the Configured Model List
    - 5.2 Verify Configuration Success
6. Other Connection Methods
    - 6.1 CLI Command Configuration (Suitable for Beginners)
    - 6.2 Modify Configuration File (Recommended Method)
        - Configuration File Location
        - Edit Configuration File
7. Common Problems and Solutions

1. Course Content

- This course will provide a detailed introduction to configuring AI model API-KEYs for OpenClaw, including the API-KEY application process, two configuration methods (CLI command configuration and configuration file modification), and how to verify successful configuration. After completing this course, you will be able to successfully connect and use various AI large model service providers.
- Master starting openclaw and setting it to auto-start on boot and disabling it

Course Objectives

- Understand the types of model service providers supported by OpenClaw
- Master the method for applying for API-KEY
- Learn how to view and verify the status of connected models

2. Preparation

> **Important**
>
> - **In the subsequent OpenClaw chapters, the OpenClaw gateway service must be started first.** The recommended way is to set it to auto-start on boot while studying the OpenClaw-related chapters, so you do not need to start the service manually every time. When it is not needed, turn off auto-start on boot, because the OpenClaw service will occupy the motherboard's memory resources in the background.

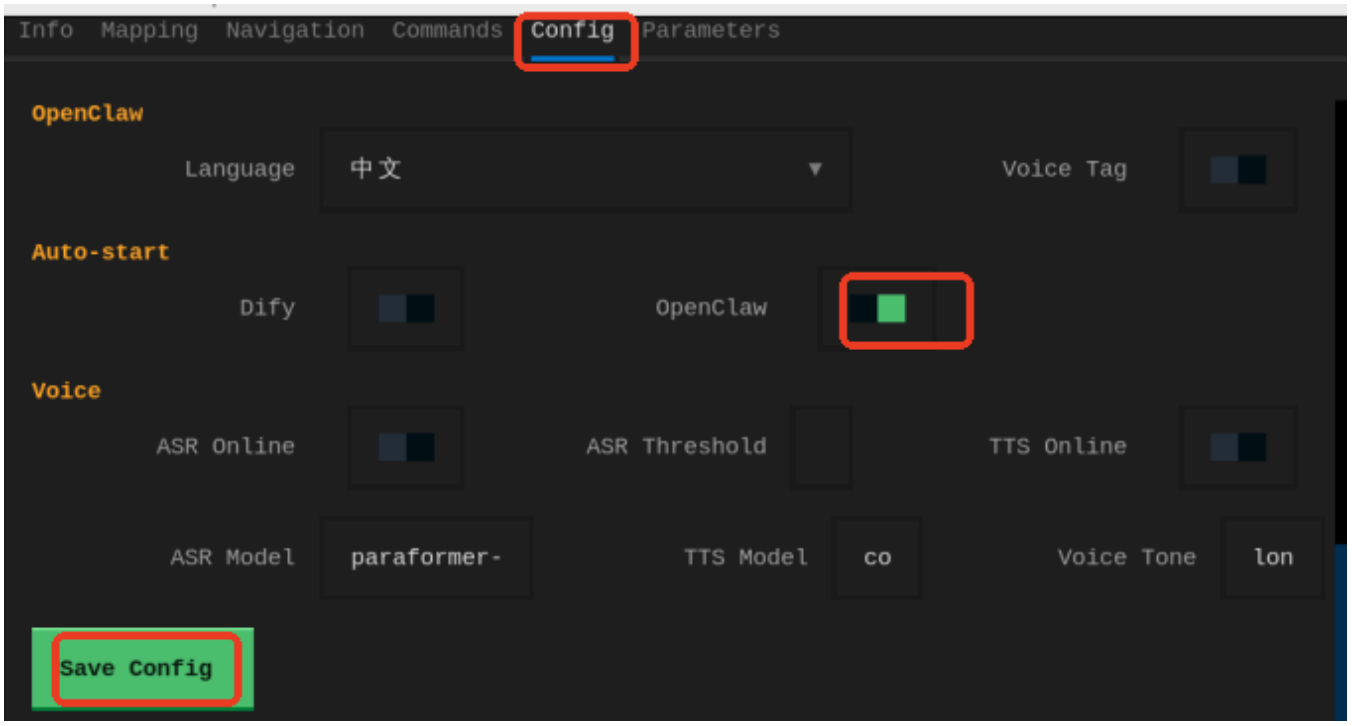
- Start the OpenClaw gateway service

```
openclaw gateway run --port 18789
```

- Set and disable OpenClaw auto-start on boot

```
robot_manage gui
```

- Turn the OpenClaw auto-start option on or off to enable/disable the auto-start-on-boot feature



3. API-KEY Application

- For API-KEY application, refer to the accompanying tutorial 【AI Large Model Development】— 【Register Model Service Account】
- OpenClaw supports many model service providers, for example:
  - **OpenAI**
  - **Alibaba Cloud** - Tongyi Qianwen
  - **DeepSeek**
  - **Zhipu AI**
  - **Moonshot**
- You can freely choose a model service provider based on the network policies of your country and region, and register and apply for an API-KEY on the corresponding official website

4. Configure OpenClaw API-KEY

4.1 Configuration Methods

OpenClaw provides two methods for configuring API-KEY:

1. **CLI Command Configuration** (suitable for beginners)
2. **Direct Configuration File Modification** (suitable for developers)
3. Configuration file path: `$HOME/.openclaw/openclaw.json`

4.2 Quick Configuration

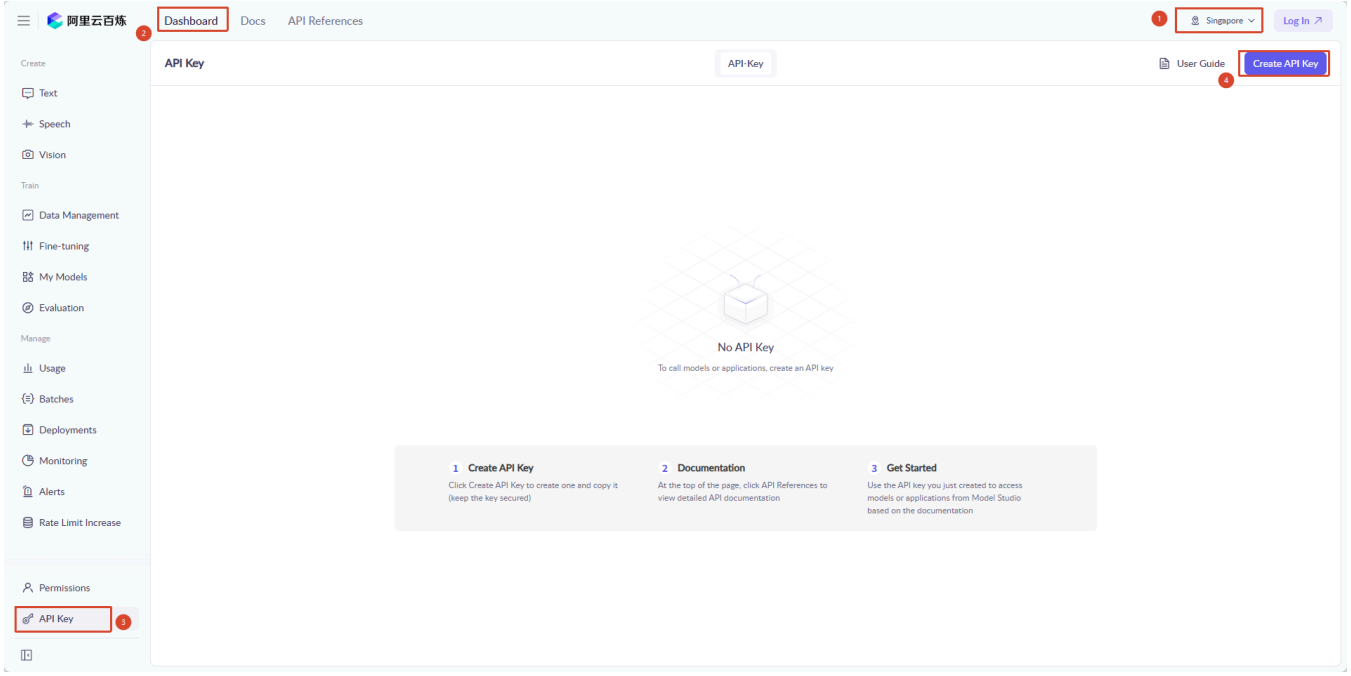
> **Note**
>
> - The factory-preset **openclaw.json** configuration file has built-in Alibaba Cloud Bailian platform's baseurl and commonly used models for quick configuration

- Suitable for quick hands-on experience. First register for an Alibaba Cloud Bailian large model account and create an API-KEY (refer to the accompanying tutorial 【AI Large Model Development】—【Register Model Service Account】). The factory preset configuration file has already configured the Bailian model parameters; just fill in the API-KEY and baseUrl to use directly.
- Open the OpenClaw configuration file:

```
nano $HOME/.openclaw/openclaw.json
```

4.2.1 Configure baseurl

- Users need to configure the baseUrl corresponding to their own account. Go to the Alibaba Cloud Model Studio Console, apply for the API key, copy the API key and the OpenAI-compatible address (Base URL), and fill them in the corresponding positions.





4.2.2 Fill in apiKey and baseUrl

- Find the corresponding fields and replace the baseurl and apiKey fields

```
"models": {
  "mode": "merge",
  "providers": {
    "bailian": {
      "baseUrl": "https://dashscope.aliyuncs.com/compatible-mode/v1",
      "apiKey": "sk-a87e1137b17b4a35af13aa4a12ac3473",
```

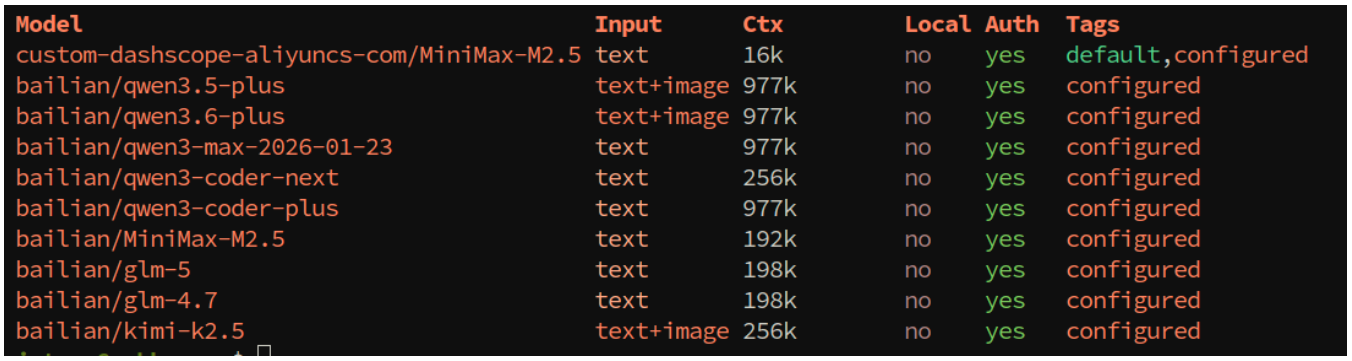
- After filling them in, press `Ctrl+X`, then press `Y` to confirm saving, and press Enter to exit.

5. View Connected Model Status

5.1 View the Configured Model List

After configuration, you can use the following command to view the connected models:

```
openclaw models list
```



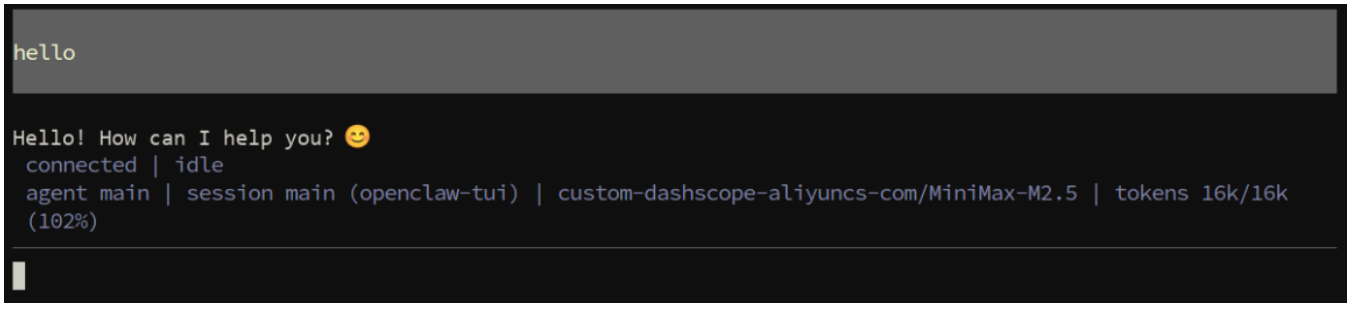
Model	Input	Ctx	Local	Auth	Tags
custom-dashscope-aliyuncs-com/MiniMax-M2.5	text	16k	no	yes	default,configured
bailian/qwen3.5-plus	text+image	977k	no	yes	configured
bailian/qwen3.6-plus	text+image	977k	no	yes	configured
bailian/qwen3-max-2026-01-23	text	977k	no	yes	configured
bailian/qwen3-coder-next	text	256k	no	yes	configured
bailian/qwen3-coder-plus	text	977k	no	yes	configured
bailian/MiniMax-M2.5	text	192k	no	yes	configured
bailian/glm-5	text	198k	no	yes	configured
bailian/glm-4.7	text	198k	no	yes	configured
bailian/kimi-k2.5	text+image	256k	no	yes	configured

5.2 Verify Configuration Success

1. **Test Conversation**

   ```
   openclaw tui
   ```

   Start the TUI interface, send a test message (such as "Hello"), and confirm the model can respond normally.



6. Other Connection Methods

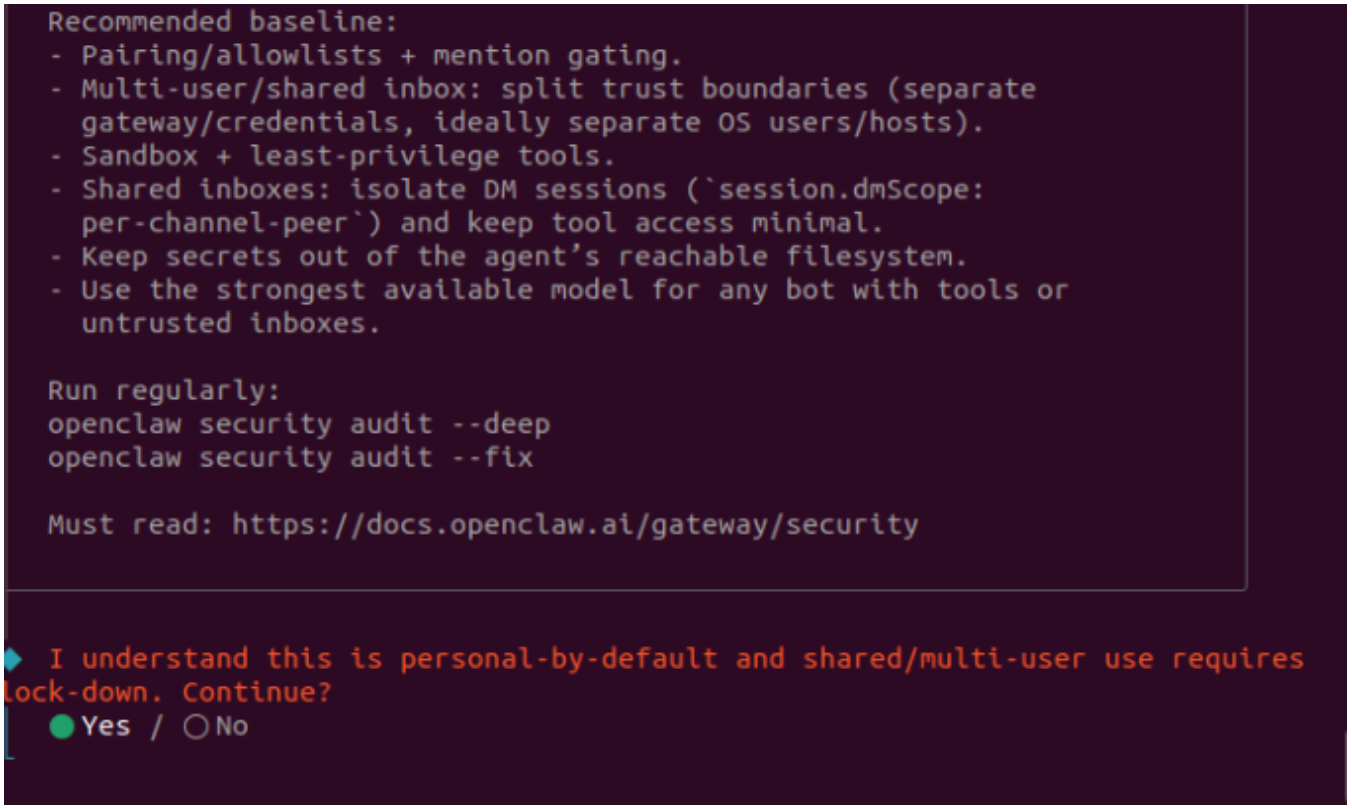
- If you have your own model provider or need to connect to other model providers later, refer to the following tutorials

6.1 CLI Command Configuration (Suitable for Beginners)

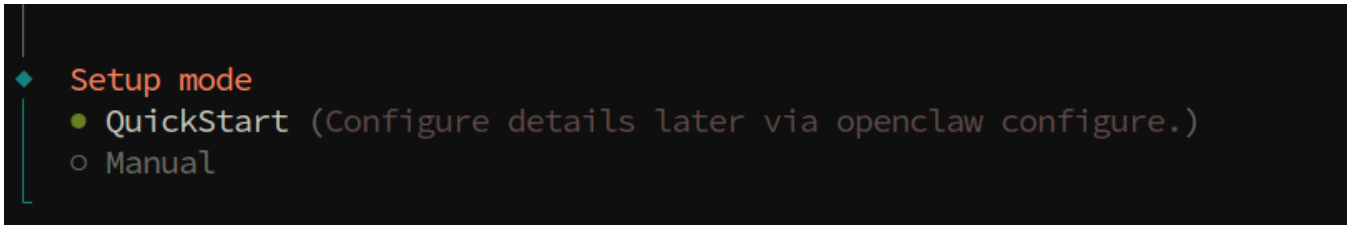
1. **Start Configuration Wizard**

   ```
   openclaw onboard
   ```

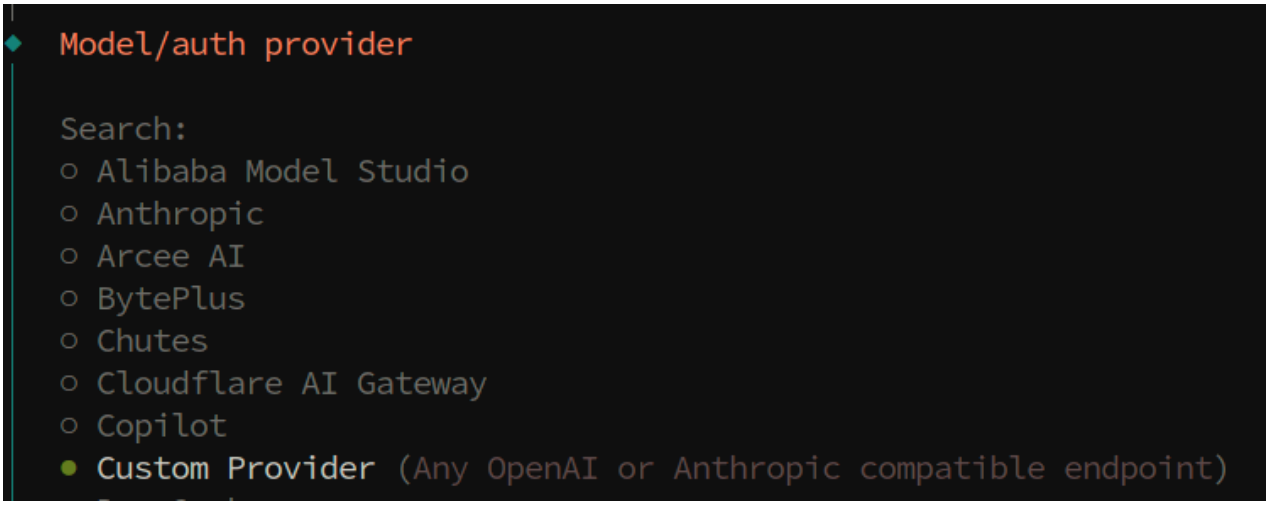
- Select 【Yes】, and press Enter



- Select configuration mode as QuickStart, and press Enter



2. **Select Model Service Provider**
    - Use the up and down arrows to select the service provider for which you have applied for an API-KEY, and press Enter to confirm. Here we take the Alibaba Cloud Bailian model provider as an example; the configuration method for other model providers is the same.
    - Here select custom provider **【Custom Provider】**

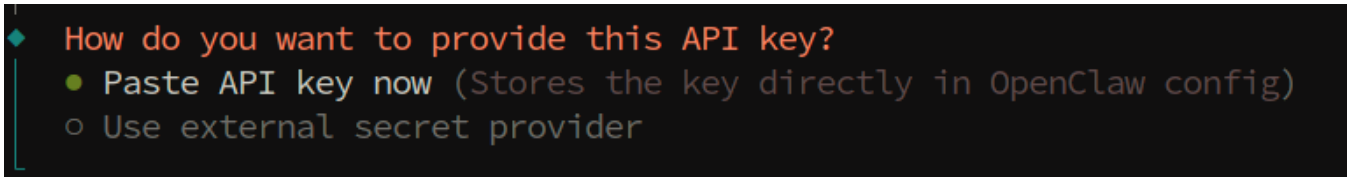


    - Fill in the base_url of the model service provider

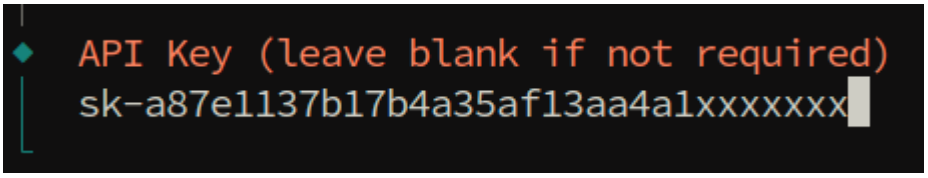


3. Enter API-KEY

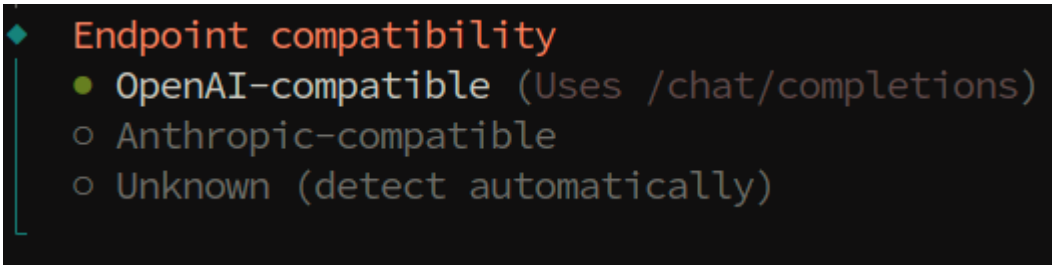
- After selecting a service provider, the system will prompt you to enter the corresponding API-KEY



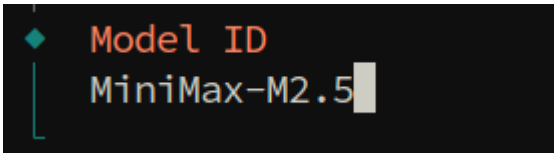
- Paste or enter your applied API-KEY and press Enter to confirm



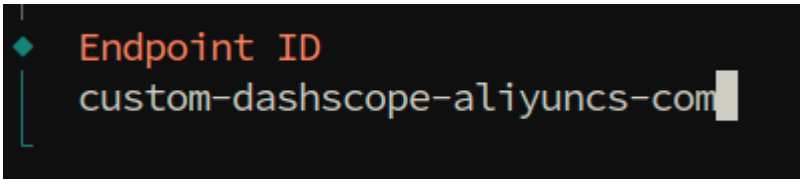
- Select interface protocol as **OpenAI-compatible**



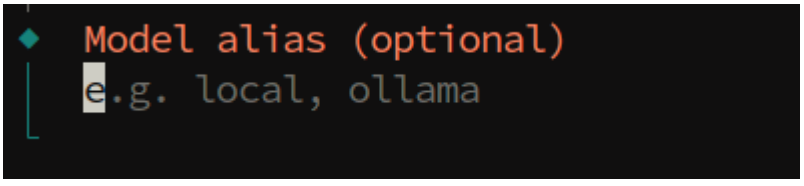
4. Fill in the model name. Here we take MiniMax as an example.



- **Endpoint ID** is the model provider identifier; you can fill it in freely



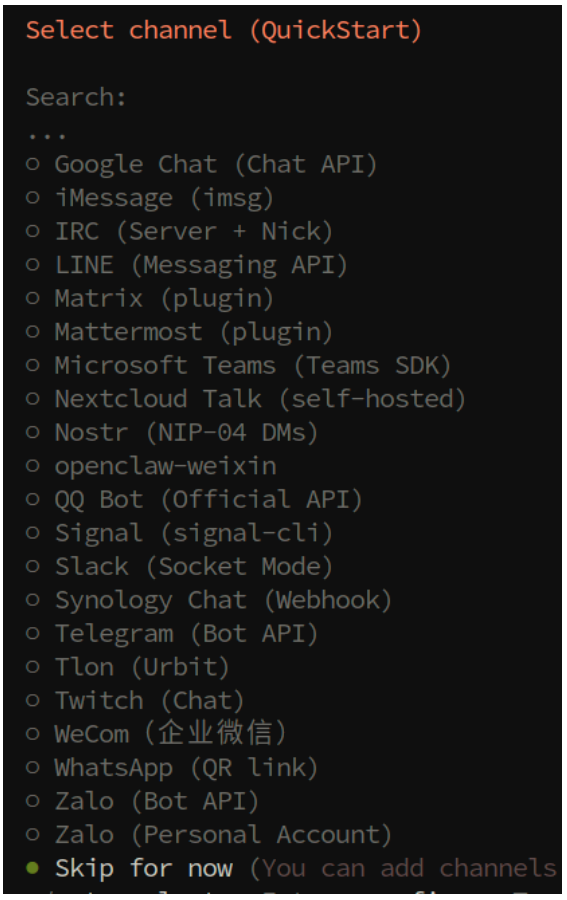
- Model alias (optional) can be left empty



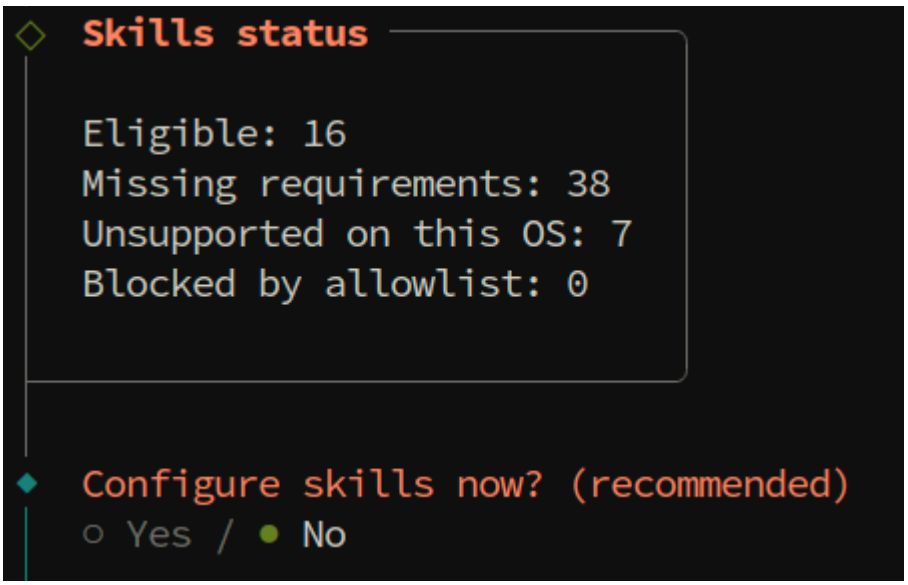
5. **Other Configuration**

- For the other configuration items, select skip for all as shown in the figure below; no configuration

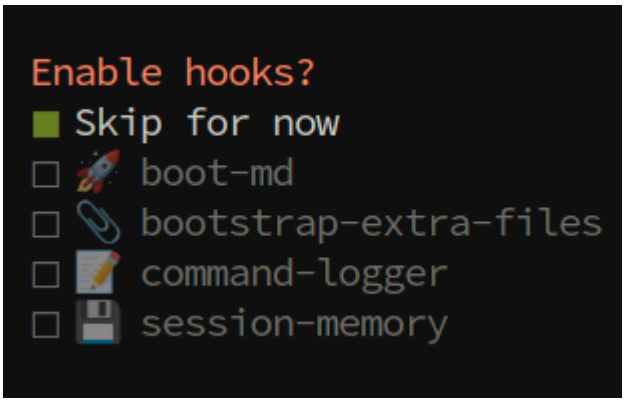
needed


Select channel (QuickStart)
Search:
...
○ Google Chat (Chat API)
○ iMessage (imsg)
○ IRC (Server + Nick)
○ LINE (Messaging API)
○ Matrix (plugin)
○ Mattermost (plugin)
○ Microsoft Teams (Teams SDK)
○ Nextcloud Talk (self-hosted)
○ Nostr (NIP-04 DMs)
○ openclaw-weixin
○ QQ Bot (Official API)
○ Signal (signal-cli)
○ Slack (Socket Mode)
○ Synology Chat (Webhook)
○ Telegram (Bot API)
○ Tlon (Urbit)
○ Twitch (Chat)
○ WeCom (企业微信)
○ WhatsApp (QR link)
○ Zalo (Bot API)
○ Zalo (Personal Account)
● Skip for now (You can add channels

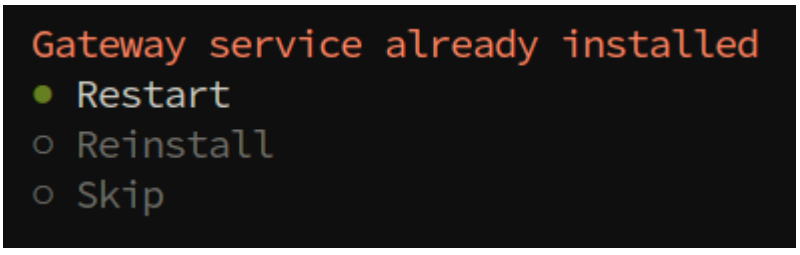

Search provider
Search:
○ Brave Search
○ Exa Search
○ Firecrawl Search
○ Gemini (Google Search)
○ Grok (xAI)
○ Kimi (Moonshot)
○ Ollama Web Search
○ Perplexity Search
○ SearXNG Search
○ Tavily Search
● Skip for now (Configure later with openclaw configure --section web)



- For Enable Hooks, press space to select **Skip for now**



- Select Restart to restart the gateway and apply the configuration



6.2 Modify Configuration File (Recommended Method)

Configuration File Location

- The configuration file is located at: `$HOME/.openclaw/openclaw.json`

Edit Configuration File

```
nano $HOME/.openclaw/openclaw.json
# Or use your favorite editor, such as vim, etc. It is recommended to modify through IDEs like vscode for easier operation
```

1. **Add Model Provider Configuration**

Add or modify the `models.providers` section in the configuration file. Refer to the format below. You can refer to the factory default **bailian** provider's configuration format:

```
{
  "models": {
    "providers": {
      "openai": {
        "apiKey": "OPENAI_API_KEY"
      },
      "anthropic": {
        "apiKey": "ANTHROPIC_API_KEY"
      },
      "google": {
        "apiKey": "GOOGLE_API_KEY"
      }
    }
  },
  "agents": {
    "defaults": {
      "model": {
        "primary": "openai/gpt-4"
      }
    }
  }
}
```



3. **Save and Exit**
    - Press `Ctrl+O` to save, `Ctrl+X` to exit (nano editor)
4. **Factory Default Bailian Configuration (Can be directly modified or referenced)**

```
"bailian": {
  "baseUrl": "https://dashscope.aliyuncs.com/compatible-mode/v1",
  "apiKey": "sk-a87e1137b17b4a35af13aaxxxxxxx",
  "api": "openai-completions",
  "models": [
    {
      "id": "qwen3.5-plus",
      "name": "qwen3.5-plus",
      "api": "openai-completions",
      "reasoning": false,
      "input": [
        "text",
        "image"
      ],
      "cost": {
        "input": 0,
```

```
        "output": 0,
        "cacheRead": 0,
        "cacheWrite": 0
      },
      "contextWindow": 1000000,
      "maxTokens": 65536
    },
    {
      "id": "qwen3.6-plus",
      "name": "qwen3.6-plus",
      "api": "openai-completions",
      "reasoning": false,
      "input": [
        "text",
        "image"
      ],
      "cost": {
        "input": 0,
        "output": 0,
        "cacheRead": 0,
        "cacheWrite": 0
      },
      "contextWindow": 1000000,
      "maxTokens": 65536
    },
    {
      "id": "qwen3-coder-next",
      "name": "qwen3-coder-next",
      "api": "openai-completions",
      "reasoning": false,
      "input": [
        "text"
      ],
      "cost": {
        "input": 0,
        "output": 0,
        "cacheRead": 0,
        "cacheWrite": 0
      },
      "contextWindow": 262144,
      "maxTokens": 65536
    },
    {
      "id": "MiniMax-M2.5",
      "name": "MiniMax-M2.5",
      "api": "openai-completions",
      "reasoning": false,
      "input": [
        "text"
      ],
      "cost": {
        "input": 0,
        "output": 0,
```

```
        "cacheRead": 0,
        "cacheWrite": 0
      },
      "contextWindow": 196608,
      "maxTokens": 32768
    },
    {
      "id": "glm-5",
      "name": "glm-5",
      "api": "openai-completions",
      "reasoning": false,
      "input": [
        "text"
      ],
      "cost": {
        "input": 0,
        "output": 0,
        "cacheRead": 0,
        "cacheWrite": 0
      },
      "contextWindow": 202752,
      "maxTokens": 16384,
      "compat": {
        "thinkingFormat": "qwen"
      }
    },
    {
      "id": "glm-4.7",
      "name": "glm-4.7",
      "api": "openai-completions",
      "reasoning": false,
      "input": [
        "text"
      ],
      "cost": {
        "input": 0,
        "output": 0,
        "cacheRead": 0,
        "cacheWrite": 0
      },
      "contextWindow": 202752,
      "maxTokens": 16384,
      "compat": {
        "thinkingFormat": "qwen"
      }
    },
    {
      "id": "kimi-k2.5",
      "name": "kimi-k2.5",
      "api": "openai-completions",
      "reasoning": false,
      "input": [
        "text",
```

```
          "image"
        ],
        "cost": {
          "input": 0,
          "output": 0,
          "cacheRead": 0,
          "cacheWrite": 0
        },
        "contextWindow": 262144,
        "maxTokens": 32768,
        "compat": {
          "thinkingFormat": "qwen"
        }
      }
    ]
  }
```

7. Common Problems and Solutions

Problem: After configuration, it prompts API-KEY is invalid or authentication failed

1. Confirm the API-KEY is correct (note there should be no extra spaces)
2. Check if the API-KEY has expired or been disabled
3. Confirm if the account balance is sufficient (some service providers require prepaid balance)

Problem: Configuration is correct but cannot connect to the model service

1. Check if the network connection is normal
2. Confirm if the network in your country/region can access the service
3. Try switching to another model service provider for testing